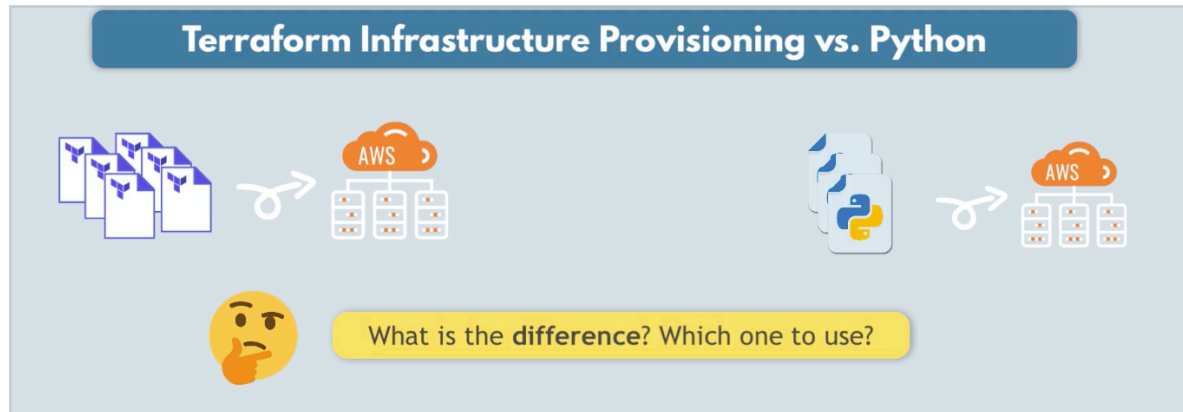


Module 14 - chapter 4

Terraform vs Python - understand when to use which tool

TF advantages for infrastructure provisioning



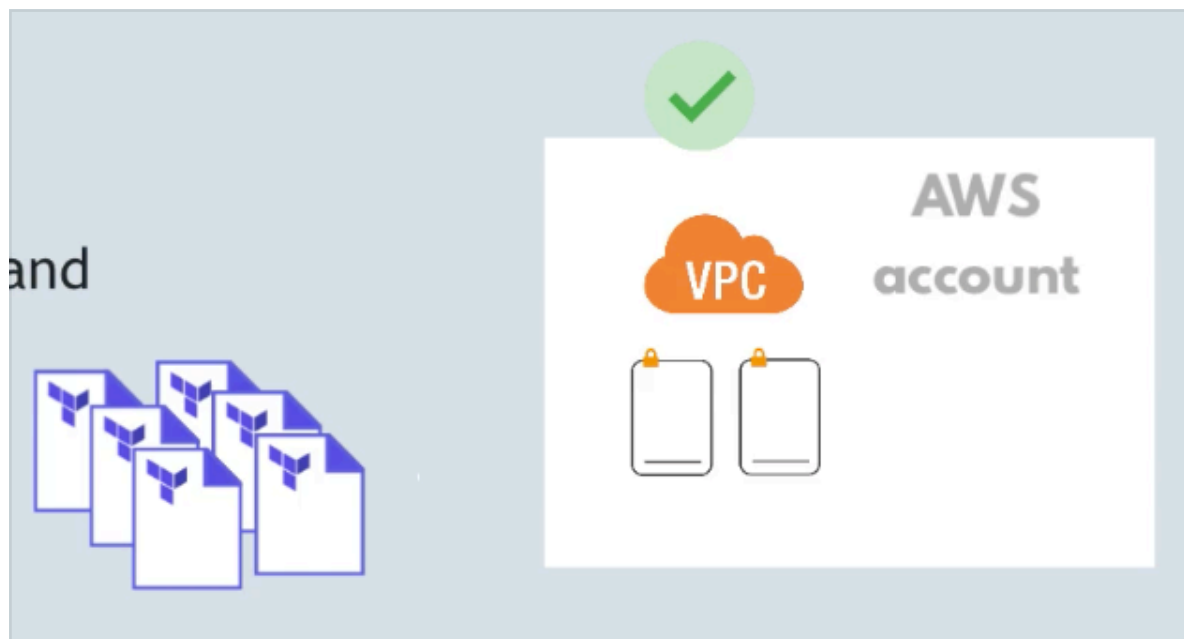
Terraform

► Terraform manages state of the infrastructure

- ✓ TF knows the current state
- ✓ TF knows the difference between current state and your configured/desired state
- ✓ TF is idempotent

Multiple executions of same config file, will always result in same end result

So if I have one VPC with 2 subnets and I execute TF 100 times the result is the same -> one VPC with 2 subnets in the AWS account



Python

When I executed my python script in the Module 14 - chapter 3 several times python didn't track the state of the infra and instead created another VPC every time I executed the script until I reach the limit of 5 vpcs at which point I got an error.

```
Run main x
/Users/nana/PycharmProjects/my-python-project/venv/bin/python /Users/nana/PycharmProjects/my-py
vpc-0d2b0f435a69078c2
{'State': 'associated'}
vpc-0c7266fef50843662
{'State': 'associated'}
vpc-05e4d423ed8512e99
{'State': 'associated'}
Process finished with exit code 0
```

VPC VPC VPC

Above the script has been run 3 times -> now there are 3 vpcs -> NOT **IDEMPOTENT**

And in the AWS console we can see that we have lots of subnets (2 per vpc)

VPC dashboard

EC2 Global View

Filter by VPC: Select a VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

Endpoints

Subnets (6/9) Info

Find resources by attribute or tag

Name	Subnet ID	State	VPC	IPv4 CIDR
-	subnet-097340146a32a0110	Available	vpc-05e4d423ed8512e99	172.31.0.0/20
-	subnet-00b2d3713691f9f54	Available	vpc-05e4d423ed8512e99	172.31.16.0/20
-	subnet-0b06b2918823f63b9	Available	vpc-05e4d423ed8512e99	172.31.0.0/20
-	subnet-0b9dc0a8295e1d5b9	Available	vpc-0c7266fef50843662 my-vpc	10.0.2.0/24
-	subnet-00666f83efde8f5c5	Available	vpc-0d2b0f435a69078c2 my-vpc	10.0.2.0/24
-	subnet-03c925f14a70b9278	Available	vpc-05c75f863074d4397 my-vpc	10.0.2.0/24
-	subnet-002f58e72a3d2db0d	Available	vpc-0c7266fef50843662 my-vpc	10.0.1.0/24
-	subnet-09d9e3c447ed05056	Available	vpc-05c75f863074d4397 my-vpc	10.0.1.0/24
-	subnet-07eb293adafc3e3d2	Available	vpc-0d2b0f435a69078c2 my-vpc	10.0.1.0/24

Subnets: subnet-0b9dc0a8295e1d5b9, subnet-002f58e72a3d2db0d, subnet-00666f83efde8f5c5, subnet-03c925f14a70b9278, subnet-09d9e3c447ed05056, subnet-07eb293adafc3e3d2

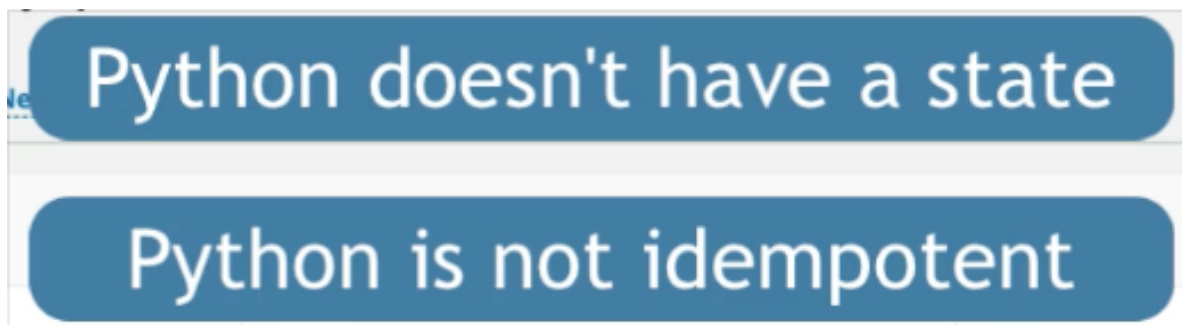
And 3 vpcs

Your VPCs (1/4) Info

Search

Name	VPC ID	State	IPv4 CIDR	IPv6 CIDR
-	vpc-05e4d423ed8512e99	Available	172.31.0.0/16	-
my-vpc	vpc-0c7266fef50843662	Available	10.0.0.0/16	-
my-vpc	vpc-05c75f863074d4397	Available	10.0.0.0/16	-
my-vpc	vpc-0d2b0f435a69078c2	Available	10.0.0.0/16	-

Terraform has State - Python doesn't have state



Removing the infra

For terraform was easy -> terraform destroy

Or delete the resources from the tf file and run -> terraform apply and tf will give me my new desired state

In Terraform you declared the end result

```
main.py
> External Libraries
Scratches and Consoles

7 new_vpc = ec2_resource.create_vpc(
8     CidrBlock="10.0.0.0/16"
9 )
10 new_vpc.create_subnet(
11     CidrBlock="10.0.1.0/24"
12 )
13 new_vpc.create_subnet(
14     CidrBlock="10.0.2.0/24"
15 )
16 new_vpc.create_tags(
17     Tags=[
18         {
19             'Key': 'Name',
20             'Value': 'my-vpc'
21         },
22     ]
23 )
24
```

But for python we need to explicitly delete the resources

In Python you need to explicitly write code to delete the resources

```
> External Libraries
Scratches and Consoles

4 ec2_resource = boto3.resource('ec2', region_name="eu-west-1")

8     CidrBlock="10.0.0.0/16"
9 )
10 new_vpc.create_subnet(
11     CidrBlock="10.0.1.0/24"
12 )
13 new_vpc.create_subnet(
14     CidrBlock="10.0.2.0/24"
15 )
16 new_vpc.create_tags(
17     Tags=[
18         {
19             'Key': 'Name',
20             'Value': 'my-vpc'
```

Using a loop and delete one by one and it is complex. For example deleting a vpc first we need to delete the resources the vpc uses like subnets and route table associations, so it is easier to go to the AWS console and delete it manually like I did in Module 14 - chapter 3

Delete VPC

✔ Will be deleted

This VPC will be deleted permanently and cannot be recovered later:

Name	VPC ID	State
my-vpc	vpc-0c7266fef50843662	✔ Available

✔ Will also be deleted

The following 2 resources will also be deleted permanently and cannot be recovered later:

Name	Resource ID	State
-	subnet-0b9dc0a8295e1d5b9	✔ Availab
-	subnet-002f58e72a3d2db0d	✔ Availab

To confirm deletion, type *delete* in the field:

Cancel
Delete

So when creating infrastructure in AWS and managing it, it is an advantage to have STATE.

Another advantage of Tf is that it is higher level -> easier to write

- ✔ TF knows the current state
- ✔ TF knows the difference between current state and your configured/desired state
- ✔ TF is idempotent
- ✔ TF is more high-level

Different Use Cases for Boto



Why should I use Python Boto3 then?

We can do things that we can't with terraform.

Different Use Cases for Boto3

- ✓ With Boto3 you can do more things, because of its low-level API
- ✓ More complex logic possible

Python is a programming language!

Boto is an AWS library



Monitoring

Backups

Scheduled Tasks



Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

We can easily add a web interface to the python project to execute tasks like click a button and do a backup:

Monitoring

Backups

Scheduled Tasks



Add Web Interface

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

Or display the health status of the app or monitoring results of our AWS infra in a user friendly UI.

When to use which tool?

Why you should prefer one over the other?



Differentiates Senior from Junior



Senior



Junior

- ✓ Do comparison between multiple tools
- ✓ Choose best tool for specific job